

Introduction to Heap - Data Structure and Algorithm Tutorials

Last Updated : 21 May, 2025



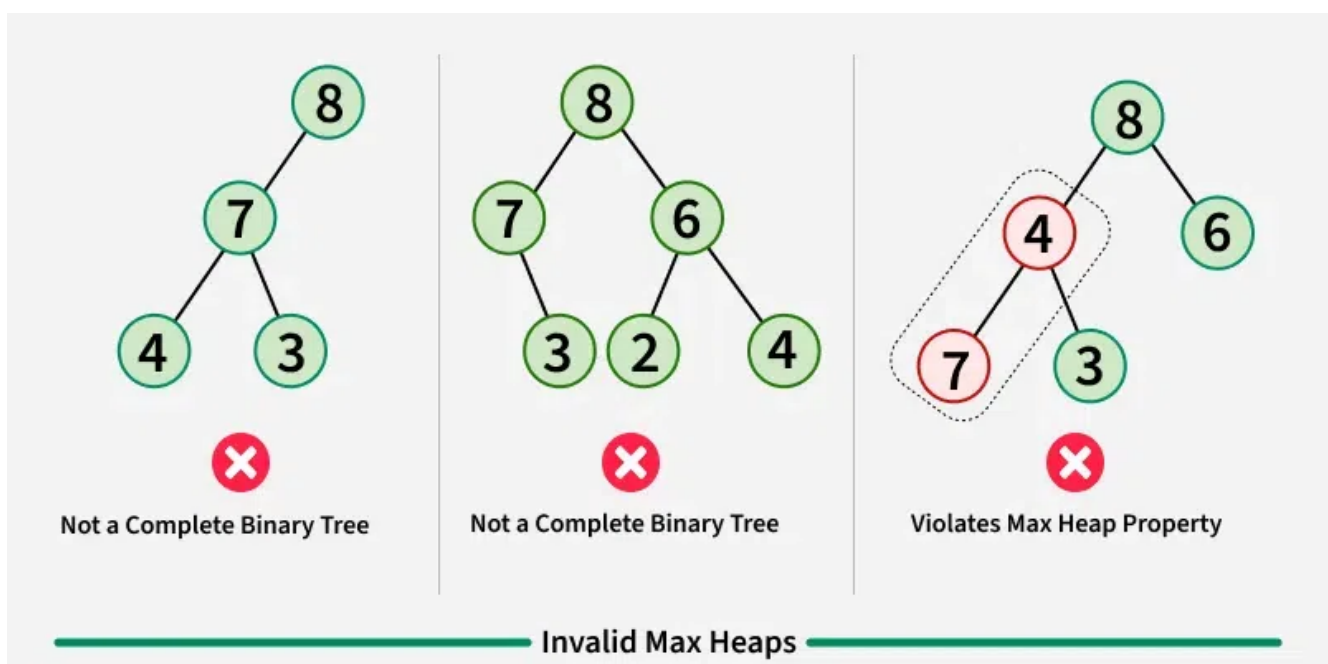
A **Heap** is a special tree-based data structure with the following properties:

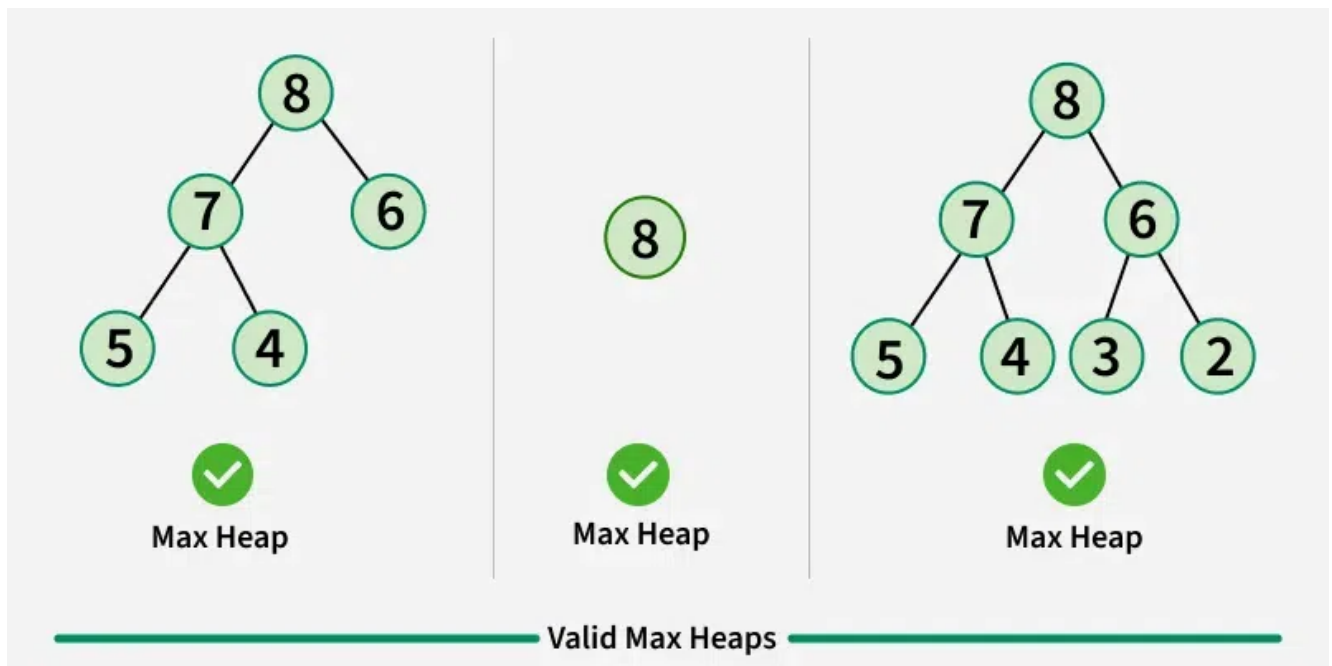
- It is a **complete binary tree** (all levels are fully filled except possibly the last, which is filled from left to right).
- It satisfies either the **max-heap property** (every parent node is greater than or equal to its children) or the **min-heap property** (every parent node is less than or equal to its children).

Note: This definition applies specifically to **binary heaps**. Other types of heaps (like Fibonacci heaps or binomial heaps) may not be complete binary trees but still maintain the heap property in their own way. So if you mean **binary heap**, your original statement is correct and standard.

Max-Heap

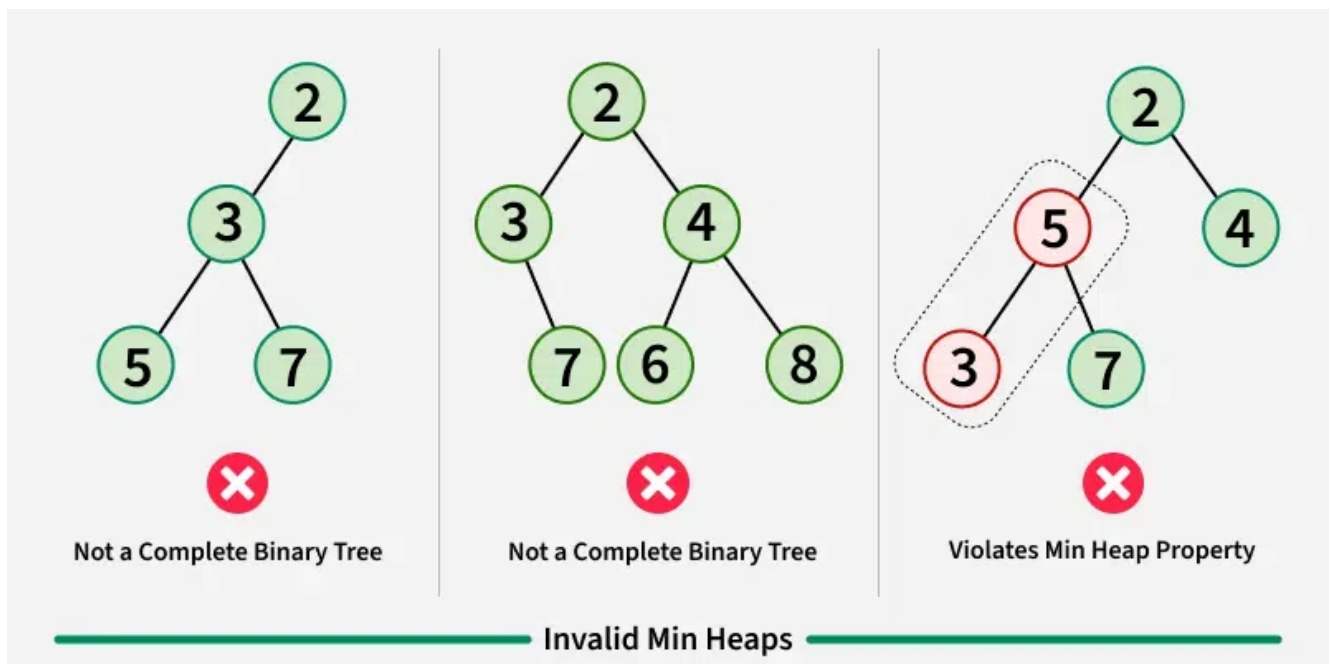
The value of the root node must be the greatest among all its descendant nodes and the same thing must be done for its left and right sub-tree also.

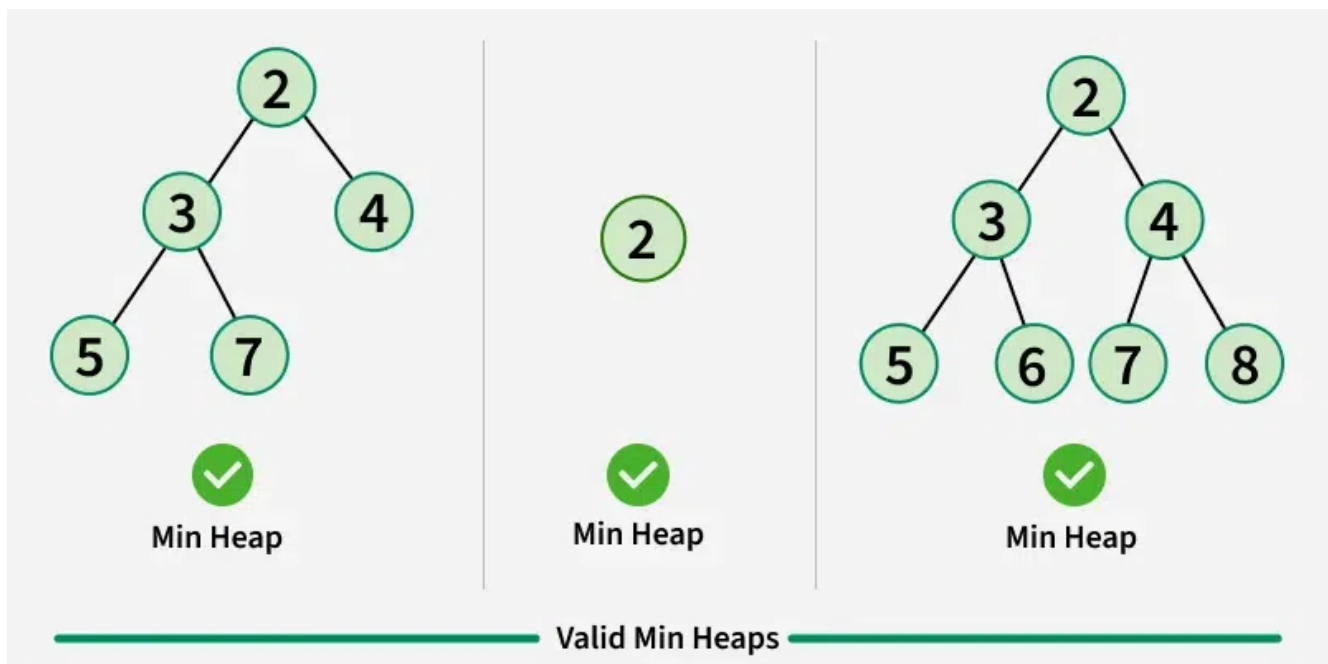




Min-Heap

The value of the root node must be the smallest among all its descendant nodes and the same thing must be done for its left and right sub-tree also.





Properties of Heap:

- The minimum or maximum element is always at the root of the heap, allowing constant-time access.
- The relationship between a parent node at index ' i ' and its children is given by the formulas: left child at index $2i+1$ and right child at index $2i+2$ for 0-based indexing of node numbers.
- As the tree is complete binary, all levels are filled except possibly the last level. And the last level is filled from left to right.
- When we insert an item, we insert it at the last available slot and then rearrange the nodes so that the heap property is maintained.
- When we remove an item, we swap root with the last node to make sure either the max or min item is removed. Then we rearrange the remaining nodes to ensure heap property (max or min)

Operations Supported by Heap:

Operations supported by **min - heap** and **max - heap** are same. The difference is just that min-heap contains minimum element at root of the tree and max - heap

contains maximum element at the root of the tree.

Heapify: It is the process to rearrange the elements to maintain the property of heap data structure. It is done when root is removed (we replace root with the last node and then call heapify to ensure that heap property is maintained) or heap is built (we call heapify from the last internal node to root) to make sure that the heap property is maintained. This operation also takes $O(\log n)$ time.

- For **max-heap**, it makes sure the maximum element is the root of that binary tree and all descendants also follow the same property.
- For **min-heap**, it balances in such a way that the minimum element is the root and all descendants also follow the same property.

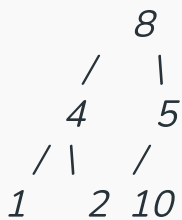
Insertion: When a new element is inserted into the heap, it can disrupt the heap's properties. To restore and maintain the heap structure, a heapify operation is performed. This operation ensures the heap properties are preserved and has a time complexity of $O(\log n)$.

Examples:

*Assume initially heap(taking **max-heap**) is as follows*



Now if we insert 10 into the heap



After repeatedly comparing with the parent nodes and swapping if required, the final heap will be look like this



Deletion:

- If we delete the element from the heap it always deletes the root element of the tree and replaces it with the last element of the tree.
- Since we delete the root element from the heap it will distort the properties of the heap so we need to perform heapify operations so that it maintains the property of the heap.

It takes **$O(\log n)$** time.

Example:

Assume initially heap(taking max-heap) is as follows



Now if we delete 15 into the heap it will be replaced by leaf node of the tree for temporary.



After heapify operation final heap will be look like this



getMax (For max-heap) or getMin (For min-heap):

It finds the maximum element or minimum element for **max-heap** and **min-heap** respectively and as we know minimum and maximum elements will always be the root node itself for min-heap and max-heap respectively. It takes **$O(1)$** time.

removeMin or removeMax:

This operation returns and deletes the maximum element and minimum element from the max-heap and min-heap respectively. In short, it deletes the root element of the heap binary tree.

Implementation of Heap Data Structure:-

The following code shows the implementation of a **max-heap**.

Let's understand the **maxHeapify** function in detail:-

maxHeapify is the function responsible for restoring the property of the Max Heap. It arranges the node **i**, and its subtrees accordingly so that the heap property is maintained.

1. Suppose we are given an array, **arr[]** representing the complete binary tree. The left and the right child of **ith** node are in indices **2*i+1** and **2*i+2**.
2. We set the index of the current element, **i**, as the 'MAXIMUM'.
3. If **arr[2 * i + 1] > arr[i]**, i.e., the left child is larger than the current value, it is set as 'MAXIMUM'.
4. Similarly if **arr[2 * i + 2] > arr[i]**, i.e., the right child is larger than the current value, it is set as 'MAXIMUM'.
5. Swap the 'MAXIMUM' with the current element.
6. Repeat steps **2 to 5** till the property of the heap is restored.

C++ Java Python JavaScript

```
// C++ code to depict
// the implementation of a max heap.

#include <bits/stdc++.h>
using namespace std;

// A class for Max Heap.
class MaxHeap {
    // A pointer pointing to the elements
    // in the array in the heap.
    int* arr;

    // Maximum possible size of
    // the Max Heap.
```

```
int maxSize;

// Number of elements in the
// Max heap currently.
int heapSize;

public:
    // Constructor function.
    MaxHeap(int maxSize);

    // Heapifies a sub-tree taking the
    // given index as the root.
    void MaxHeapify(int);

    // Returns the index of the parent
    // of the element at ith index.
    int parent(int i)
    {
        return (i - 1) / 2;
    }

    // Returns the index of the left child.
    int lChild(int i)
    {
        return (2 * i + 1);
    }

    // Returns the index of the
    // right child.
    int rChild(int i)
    {
        return (2 * i + 2);
    }

    // Removes the root which in this
    // case contains the maximum element.
    int removeMax();

    // Increases the value of the key
    // given by index i to some new value.
    void increaseKey(int i, int newVal);
```

```

// Returns the maximum key
// (key at root) from max heap.
int getMax()
{
    return arr[0];
}

int curSize()
{
    return heapSize;
}

// Deletes a key at given index i.
void deleteKey(int i);

// Inserts a new key 'x' in the Max Heap.
void insertKey(int x);
};

// Constructor function builds a heap
// from a given array a[]
// of the specified size.
MaxHeap::MaxHeap(int totSize)
{
    heapSize = 0;
    maxSize = totSize;
    arr = new int[totSize];
}

// Inserting a new key 'x'.
void MaxHeap::insertKey(int x)
{
    // To check whether the key
    // can be inserted or not.
    if (heapSize == maxSize) {
        cout << "\n0verflow: Could not insertKey\n";
        return;
    }

    // The new key is initially
    // inserted at the end.
    heapSize++;

```



```

int i = heapSize - 1;
arr[i] = x;

// The max heap property is checked
// and if violation occurs,
// it is restored.
while (i != 0 && arr[parent(i)] < arr[i]) {
    swap(arr[i], arr[parent(i)]);
    i = parent(i);
}
}

// Increases value of key at
// index 'i' to new_val.
void MaxHeap::increaseKey(int i, int newVal)
{
    arr[i] = newVal;
    while (i != 0 && arr[parent(i)] < arr[i]) {
        swap(arr[i], arr[parent(i)]);
        i = parent(i);
    }
}

// To remove the root node which contains
// the maximum element of the Max Heap.
int MaxHeap::removeMax()
{
    // Checking whether the heap array
    // is empty or not.
    if (heapSize <= 0)
        return INT_MIN;
    if (heapSize == 1) {
        heapSize--;
        return arr[0];
    }

    // Storing the maximum element
    // to remove it.
    int root = arr[0];
    arr[0] = arr[heapSize - 1];
    heapSize--;

```

```

// To restore the property
// of the Max heap.
MaxHeapify(0);

return root;
}

// In order to delete a key
// at a given index i.
void MaxHeap::deleteKey(int i)
{
    // It increases the value of the key
    // to infinity and then removes
    // the maximum value.
    increaseKey(i, INT_MAX);
    removeMax();
}

// To heapify the subtree this method
// is called recursively
void MaxHeap::MaxHeapify(int i)
{
    int l = lChild(i);
    int r = rChild(i);
    int largest = i;
    if (l < heapSize && arr[l] > arr[i])
        largest = l;
    if (r < heapSize && arr[r] > arr[largest])
        largest = r;
    if (largest != i) {
        swap(arr[i], arr[largest]);
        MaxHeapify(largest);
    }
}

// Driver program to test above functions.
int main()
{
    // Assuming the maximum size of the heap to be 15.
    MaxHeap h(15);

    // Asking the user to input the keys:

```

```

int k, i, n = 6, arr[10];
cout << "Entered 6 keys:- 3, 10, 12, 8, 2, 14 \n";
h.insertKey(3);
h.insertKey(10);
h.insertKey(12);
h.insertKey(8);
h.insertKey(2);
h.insertKey(14);

// Printing the current size
// of the heap.
cout << "The current size of the heap is "
    << h.curSize() << "\n";

// Printing the root element which is
// actually the maximum element.
cout << "The current maximum element is " << h.getMax()
    << "\n";

// Deleting key at index 2.
h.deleteKey(2);

// Printing the size of the heap
// after deletion.
cout << "The current size of the heap is "
    << h.curSize() << "\n";

// Inserting 2 new keys into the heap.
h.insertKey(15);
h.insertKey(5);
cout << "The current size of the heap is "
    << h.curSize() << "\n";
cout << "The current maximum element is " << h.getMax()
    << "\n";

return 0;
}

```

Output

```
Entered 6 keys:- 3, 10, 12, 8, 2, 14
The current size of the heap is 6
The current maximum element is 14
The current size of the heap is 5
The current size of the heap is 7
The current maximum element is 15
```

Please refer the following articles for more details about Heap Data Structure

- [Advantages and Disadvantages of Heap](#)
- [Applications of Heap Data Structure](#)

Library Implementations of Heap or Priority Queue

- [Heap in C++ STL](#)
- [priority_queue in C++](#)
- [PriorityQueue in Java](#)
- [heapq in Python](#)

 Comment

More info 

Advertise with us

Next Article 

Binary Heap

Similar Reads

Heap Data Structure

A Heap is a complete binary tree data structure that satisfies the heap property: for every node, the value of its children is greater than or equal to its own value. Heaps are usually used to implement priority...

 2 min read

Introduction to Heap - Data Structure and Algorithm Tutorials

A Heap is a special tree-based data structure with the following properties:It is a complete binary tree (all levels are fully filled except possibly the last, which is filled from left to right).It satisfies either the...

 15+ min read

Binary Heap

A Binary Heap is a complete binary tree that stores data efficiently, allowing quick access to the maximum or minimum element, depending on the type of heap. It can either be a Min Heap or a Max...

🕒 13 min read

Advantages and Disadvantages of Heap

Advantages of Heap Data Structure
Time Efficient: Heaps have an average time complexity of $O(\log n)$ for inserting and deleting elements, making them efficient for large datasets. We can convert any array to a...

🕒 2 min read

Time Complexity of building a heap

Consider the following algorithm for building a Heap of an input array A. A quick look over the above implementation suggests that the running time is $O(n * \lg(n))$ since each call to Heapify costs $O(\lg(n))$ an...

🕒 2 min read

Applications of Heap Data Structure

Heap Data Structure is generally taught with Heapsort. Heapsort algorithm has limited uses because Quicksort is better in practice. Nevertheless, the Heap data structure itself is enormously used. Priority...

🕒 2 min read

Comparison between Heap and Tree

What is Heap? A Heap is a special Tree-based data structure in which the tree is a complete binary tree. Types of Heap Data Structure: Generally, Heaps can be of two types: Max-Heap: In a Max-Heap the key...

🕒 3 min read

When building a Heap, is the structure of Heap unique?

What is Heap? A heap is a tree based data structure where the tree is a complete binary tree that maintains the property that either the children of a node are less than itself (max heap) or the children a...

🕒 4 min read

Some other type of Heap



Easy problems on Heap

